NOXISA — Complete Project State & Roadmap

Session-continuity document, consolidated through the complete ext2 write-support + crash-safety arc

Consolidated update — supersedes all prior revisions

Contents

Revision Note (This Consolidation) **3**

PART 1 — What Noxisa Is **4**

PART 2 — Ground Truth: What Is Actually Proven, Right Now **5**
- 2.1 Core kernel (proven QEMU+Dell, unchanged for many sessions) . . . 5
- 2.2 Isolation & security (proven QEMU+Dell) . . . 5
- 2.3 HAL (proven QEMU+Dell) . . . 5
- 2.4 NVMe storage (proven QEMU+Dell) . . . 5
- 2.5 GPT + ext2 filesystem — now a complete read-write filesystem with a genuine crash-safety pass . . . 5
- 2.6 Standing safety rule (Part 19.2, still in force) . . . 7
- 2.7 Current kshell command list . . . 7

PART 3 — Architecture, Design Principles, and Full Roadmap **8**
- 3.1 What we are explicitly not doing right now, and why . . . 8
- 3.2 Design principles (non-negotiable) . . . 8
- 3.3 The one roadmap (supersedes all prior orderings) . . . 8
- 3.4 Privacy & security primitives — reference table . . . 9
- 3.5 Directory structure — present-day . . . 9

PART 4 — The Real Bugs Found and Fixed **11**
- 4.1 Condensed history (bugs 1–31) . . . 11
- 4.2 This session's additions (bugs 32–35) — the ext2 write-support arc . . . 11
- 4.3 Later additions this session (bugs 36-39) - indirect allocation, directory growth/deletion, crash-safety . . . 12

PART 5 — Established Workflow Conventions **13**

PART 6 — What's Next, In Order **14**

PART 7 — How to Keep This Document Honest **15**

PART 8 — Desktop/UI Vision (reference only, not current work) **16**

PART 9 — Condensed Historical Session Logs (Parts 9–19) **17**

PART 20 — SESSION LOG: Multi-Level Path Resolution (patch_45, QEMU confirmed) **18**

PART 21 — SESSION LOG: Singly-Indirect Block Reading (patch_46, QEMU confirmed) **18**

PART 22 — SESSION LOG: GPT CRC32 Validation (patch_47, QEMU confirmed) **18**

PART 23 — SESSION LOG: Write-In-Place (patch_48, QEMU confirmed) **18**

PART 24 — SESSION LOG: Block Allocation for Growth (patch_49, QEMU confirmed) **19**

PART 25 — SESSION LOG: Real File Creation (patch_50, QEMU confirmed) **19**

PART 26 — SESSION LOG: Real File Deletion (patch_51, QEMU confirmed) — closes the write-support arc **19**

PART 15 — Recurring Mistakes Log (cumulative, keep appending) **21**

PART 27 — SESSION LOG: Indirect-Block Allocation, Directory Growth/Deletion, and a Crash-Safety Audit (patches 52-55, QEMU confirmed) **22**

PART 28 — Patch Index (cumulative) **23**

PART 29 — Executive Summary: What's Done, What Broke and Got Fixed, What's Left **24**

What's fully done (proven QEMU+Dell, stable across many sessions) 24

What's done, QEMU-verified, Dell verification outstanding 24

What bugs/errors were found and fixed (cumulative highlights) 24

What's genuinely still remaining 24

Revision Note (This Consolidation)

This is a full consolidation of the NOXISA project-state document, merging the original `NOXISA_COMPLETE_updated.pdf` (through Part 18 — HAL Phases A/B.1, scheduler race fix), `NOXISA_Part19_update.pdf` (the Omarchy migration, the Dell NVMe incident, and HAL Phase C — GPT parsing, read-only ext2, VFS integration, `mount_root()`), and two further sessions' work: **the complete ext2 write-support arc** (patches 45–55), which took ext2 from read-only to a filesystem that can genuinely create, grow (including into indirect blocks), overwrite, and delete real files AND directories on disk, closing with a genuine crash-safety ordering audit — all verified end-to-end on real `BlockDevice` writes.

Per this project's own Part 7 rule ("don't let a second parallel doc start later — update this one in place"), this document **replaces** every prior PDF as the single authoritative reference. Paste this whole document at the start of any new session.

Headline status, for anyone skimming: the kernel has a full isolation, capability, and scheduling foundation (Parts 2–4, unchanged and still solid from earlier sessions); a working NVMe storage stack; a GPT+ext2 filesystem that is a genuinely complete read-write filesystem — file and directory creation, growth (direct and indirect blocks), overwrite, and deletion are all real, and the two orderings that could have left the filesystem in a dangerous state after a crash have been found and fixed. What's left in ext2 is narrow and named: no `mkdir`, no true atomicity/journaling, no real crash-injection test harness. The next roadmap milestone is **Networking**, not yet started. After that: Graphics/Compositor, Package Manager, Linux Compatibility Layer, Desktop, Hardening pass — see Part 6 for the full ordering and Part 29 for a one-page "what's done vs. what's left" summary.

PART 1 — What Noxisa Is

A solo-built x86-64 kernel in Rust, booted via Limine on UEFI, with its own memory manager, paging, heap, GDT/IDT/TSS, syscalls, ring-3 execution, a preemptive scheduler, a Hardware Abstraction Layer (Console/InputDevice/DisplayDevice/BlockDevice), a real NVMe storage driver, a GPT partition parser, an ext2 filesystem driver with genuine read **and** write support, a kernel shell (`kshell`), and a framebuffer with a compositor placeholder.

Core philosophy: maximum capability with minimum wasted effort — build the parts that make Noxisa unique (kernel architecture, security model, AI runtime philosophy, scheduler), and reuse proven open-source components for everything else (drivers, libraries, algorithms), after understanding and verifying them, not before.

Priority ordering: security is built before features, deliberately — isolation → allocator hardening → capabilities → VFS/IPC → drivers/storage → networking → desktop. This was a real, considered decision to avoid building IPC and networking on top of a kernel that only pretends to isolate processes or enforce permissions.

AI philosophy (separate long-term track, not started): AI is a service running on top of Noxisa, not the foundation of the OS. The OS works completely without AI. Users choose local model, cloud model, or no AI. No telemetry by default. "Wisdom AI" (the project's AI assistant concept) is deliberately set aside until the kernel/security foundation is mature.

Testing discipline: every claim about kernel state must be verified on both QEMU and real hardware (a Dell Inspiron 5502) before it counts as fully done. One deliberate, honest, standing exception: after a real NVMe write test destroyed the Dell's boot drive (Part 19.2), real-hardware write-path testing now requires a disposable external drive, not the machine's only internal one — so most of the ext2 write-support work in this document is **QEMU-verified only**, explicitly and correctly, not silently assumed to also work on the Dell.

Environment (current): Omarchy Linux (Arch-based) directly on the same Dell Inspiron 5502 previously used for hardware verification — migrated from Windows/WSL2/Rufus during the Part 19 session. QEMU 11.1.0, `edk2-ovmf` firmware, Rust `nightly-2026-04-01` with `rust-src` and the `x86_64-unknown-none` target, Limine v8.x built from source. A standing `run-qemu.sh` script boots `nexus.iso` via UEFI with a disposable `nvme-test.img` NVMe device attached.

PART 2 — Ground Truth: What Is Actually Proven, Right Now

This is the only part of this document that is fact rather than plan. Everything below is either **proven QEMU+Dell** or explicitly marked **QEMU-only**.

2.1 Core kernel (proven QEMU+Dell, unchanged for many sessions)

- **Boot:** Limine native protocol, higher-half kernel, real UEFI hardware boot, base revision 3.
- **Memory:** PMM, HHDM-backed heap, paging, `phys_to_virt`/`virt_to_phys`, plus `map_mmio()` for real PCI MMIO mapping.
- **CPU:** GDT, IDT, exception handlers, SSE, TSS with correct RSP0 + IST for `#DF`.
- **Scheduler:** preemptive, ISR-driven context switching; `kshell` genuinely running and responding to typed commands.
- **Syscalls:** STAR/LSTAR/EFER configured, real `naked-asm` syscall handler, `sys_write`/`sys_exit` working, `sys_draw` capability-gated.
- **Ring 3:** a real process executing `SYSCALL` from ring 3.

2.2 Isolation & security (proven QEMU+Dell)

Per-process page tables, W^X enforcement, guard pages, a minimal capability bitmask (`CAP_WRITE_SERIAL`, `CAP_DRAW`), VFS owner/group/other permission enforcement, default-deny IPC channels, memory scrubbing on free, kernel-level audit logging, real process teardown with frame reclamation, per-process identity (uid/gid/cwd/open_files), generic process spawning, multiple concurrent processes, and genuine preemptive scheduling (a non-cooperating infinite loop is reclaimed by the timer without ever yielding).

2.3 HAL (proven QEMU+Dell)

`Console`/`InputDevice`/`DisplayDevice`/`BlockDevice` trait wrappers, domain-specific by design (not one generic `Device` trait), registered at boot via `hal::init()`. Proven genuine pass-throughs (not just structure) via dedicated kshell tests for each.

2.4 NVMe storage (proven QEMU+Dell)

Full stack from bare PCI enumeration through a working, HAL-abstracted block device: PCI config-space enumeration, real MMIO register mapping (with a real HHDM-safety investigation behind it — base revision 3 does *not* unconditionally map a bare PCI BAR), admin queue initialization, Identify Controller/Namespace, I/O queue creation, real Read/Write NVM commands, and the `BlockDevice` HAL adapter (`NvmeBlockDevice`) over all of it.

Known, named limits: ≤4096-byte/single-PRP1 transfers only (no PRP list yet); hardcoded namespace ID 1 (no multi-namespace); no namespace-geometry caching.

2.5 GPT + ext2 filesystem — now a complete read-write filesystem with a genuine crash-safety pass

GPT partition parsing (proven QEMU, via `hal::registry::block_read()` exclusively — never touches `drivers::nvme` directly):

- Header + partition-array **CRC32 validation**, a real CRC-32/ISO-HDLC implementation — a corrupted GPT is rejected, not silently trusted.
- No backup-GPT fallback if the primary header is bad (deliberately deferred).

ext2 filesystem driver — started read-only, is now a complete read-write filesystem with real allocation, growth, creation, deletion (files and directories), and an audited crash-safety ordering pass:

Capability	Status	Patch
Superblock + block-group-descriptor parsing	DONE	42
Root-directory-only file read	DONE	42
Real VFS integration (`mount_to_vfs`, `readdir`)	DONE	43
`mount_root()` — automatic boot-time root mount	DONE	44
Multi-level path resolution (`/a/b/c`)	DONE	45
Singly-indirect block reading (files > 12 blocks)	DONE	46
GPT header + partition-array CRC32	DONE	47
Write-in-place for an existing file, within its already-allocated blocks	DONE	48
Real block allocation — growth into new direct blocks via a genuine block-bitmap scan	DONE	49
Real file creation — inode allocation + directory-entry insertion (reusing real `rec_len` slack)	DONE	50
Real file deletion — block/inode freeing + directory-entry removal, with genuine slot reuse	DONE	51
Indirect-block allocation — growth past 12 direct blocks, allocating the indirect block itself	DONE	52
Directory growth — allocating a directory an additional block once existing ones have no slack	DONE	53
Directory deletion — removing an empty directory, with correct parent link-count/used-dirs bookkeeping	DONE	54
Crash-safety ordering audit — reordered the two genuinely dangerous sequences (deletion, directory growth) so a crash can only ever leak, never dangle or misparse	DONE	55

Still explicitly, deliberately out of scope (named, not forgotten — see Part 6 for sizing if ever picked up):

- **True atomicity / a journal.** Patch 55 made every dangerous ordering safe (a crash can only ever leak a resource, never leave a dangling reference or a misparsed directory block) but individual operations are still several separate disk writes, not one atomic transaction. A crash can still interrupt a single bitmap-bit write or leave a free-count stale — cosmetic, fsck-fixable, never dangerous, but not full atomicity.
- **Real crash-injection testing.** Patch 55 was verified by code-review audit plus the full regression suite, not by an actual QEMU restart-mid-write harness — building that harness is its own separate effort.
- **Directory creation (`mkdir`).** Every test that needs a fresh directory still relies on a host-staged one; this driver can grow and delete directories but cannot create one from scratch.
- **Dell hardware verification** — the entire GPT/ext2 write-support arc (patches 45–55) is QEMU-verified only. Real-hardware write-path testing is deliberately deferred pending a disposable external test drive, per the standing safety rule below.

2.6 Standing safety rule (Part 19.2, still in force)

Never run a real block-device write test (`nvmeiotest`, `blockdevtest`, any ext2 write test) against a machine's only internal drive. This exact mistake cost one Dell boot drive already. QEMU's disposable `nvme-test.img` remains the primary iteration loop for everything write-related; real hardware write-path testing requires a separate external/spare drive.

2.7 Current kshell command list

```
help meminfo version reboot ticks uptime syscalls
scrubtest teardowntest multitest audittest heaptest
isotest isofault wxtest guardtest captest permtest
ipctest spawn restest conctest preempttest drawtest
ownertest haltest pcitest mmiotest nvmeinittest
identifytest nvmeiotest blockdevtest gpttest
ext2test ext2mounttest roottest ext2pathtest
ext2indirecttest ext2writetest ext2growtest
ext2createtest ext2deletetest ext2indirectgrowtest
ext2dirgrowtest ext2rmdirtest
```

PART 3 — Architecture, Design Principles, and Full Roadmap

3.1 What we are explicitly not doing right now, and why

Directory structure and multi-subsystem scaffolding are not built ahead of actual need. A session is judged by what the kernel can genuinely do afterward (verified QEMU, ideally +Dell), not by how much structure it grew. Restructuring is allowed only when required to build the next real capability — never as the capability itself.

3.2 Design principles (non-negotiable)

- A session is judged by verified new capability, not new structure.
- Security is not a late stage — it moved to immediately after process management.
- Don't build on top of a known-fake foundation.
- **Prefer wrapping over forking.** The hardened allocator wraps `linked_list_allocator`; VFS permissions wrap Tmpfs; HAL wraps the existing drivers; ext2's write path wraps and reuses its own read primitives (`block_pointers()`, `read_inode()`) rather than duplicating them.
- Every claim about kernel state must be verified before it moves from "planned" to "done."
- **Prefer the smaller, well-scoped, testable step over the larger architectural leap.** This governed the entire ext2 write-support arc: in-place overwrite before growth, growth before creation, creation before deletion — each its own patch, each independently QEMU-verified, none bundled together.

3.3 The one roadmap (supersedes all prior orderings)

```
Core Kernel
  → Hardened heap allocator                                  DONE
  → Real per-process isolation                                DONE
  → VFS permissions                                            DONE
  → IPC                                                         DONE
  → Memory scrubbing                                            DONE
  → Kernel-level audit logging                                  DONE
  → Real per-process teardown                                   DONE
  → Real per-PCB capability system                              DONE
  → Generic process creation                                    DONE
  → Per-process resources                                       DONE
  → Multiple concurrent user processes                          DONE
  → Preemptive scheduling                                       DONE
  → HAL + Driver Manager
      Phase A (console/input/display)                           DONE, QEMU+Dell
      Phase B.1 (NVMe storage primitive)                         DONE, QEMU+Dell
      Phase C.1 (GPT partition parsing + CRC32)                   DONE, QEMU
      Phase C.2 (read-only ext2)                                  DONE, QEMU
      Phase C.3 (ext2 ↔ VFS integration, mount_root())             DONE, QEMU
      Phase C.4 (ext2 WRITE SUPPORT, complete)                     DONE, QEMU
        multi-level paths · indirect-block reads/allocation
        · write-in-place · block allocation/growth
        · file creation/deletion · directory growth/deletion
        · crash-safety ordering audit
  → Networking                                                NEXT — not started
      (per-process socket capability, mandatory chokepoint, Tails-derived)
  → Graphics / Compositor                                     not started
      (once IPC exists for a real client/compositor protocol — it does now)
  → Package Manager                                           not started
      (signature + sandbox from day one)
  → Linux Compatibility Layer                                 not started
```

```
    (small, audited syscall allowlist)
  → Desktop                                                  not started
    (multi-repo/HAL/filesystem restructuring becomes worth doing here)
  → Hardening pass                                           not started
    (ASLR, secure boot, deeper audit)
```

3.4 Privacy & security primitives — reference table

Primitive	Source	Status
Hardened heap allocator (canary + zero-on-free)	GrapheneOS `hardened_malloc`	DONE, QEMU+Dell
Per-process page tables, W^X, guard pages	Standard OS security	DONE, QEMU+Dell
VFS owner/group/other permission check	Standard Unix model	DONE, QEMU+Dell
Capability bitmask	seL4	DONE, QEMU+Dell
Explicit-policy IPC (default-deny)	Qubes qrexec	DONE, QEMU+Dell
Memory scrubbing on free	Original design	DONE, QEMU+Dell
Kernel-level audit logging	Original design	DONE, QEMU+Dell
Preemptive scheduling	Standard scheduler design	DONE, QEMU+Dell
HAL trait wrappers over drivers	Standard OS HAL, domain-specific	DONE, QEMU+Dell
NVMe storage primitive	Redox `nvmed` (cross-check only)	DONE, QEMU+Dell
GPT parsing + CRC32	UEFI spec	DONE, QEMU
ext2 read (direct + indirect blocks, multi-level paths)	OSDev wiki / Linux `ext2.h` (cross-check only)	DONE, QEMU
ext2 write (overwrite, growth incl. indirect, create/delete files+dirs, crash-safety ordering)	Standard ext2 on-disk format	DONE, QEMU
Mandatory network chokepoint	Tails (forced Tor model)	Networking stage — not started
Per-process consent gates (network/mic/camera)	GrapheneOS granular toggles	Alongside capability system — not started
Small, audited Linux compat syscall allowlist	Original design	Compat layer stage — not started
Disposable, one-shot processes	Qubes disposable VMs	After generic spawning matures further
Secure boot chain	External research	Late
CHERI-style hardware capability pointers	Research awareness only	Not planned

3.5 Directory structure — present-day

```
nexus-os/kernel/src/
  main.rs          kernel_main_native — the only live boot path;
                    kshell command dispatch is a long else-if chain here,
                    NOT in kshell/mod.rs
  drivers/         apic, pic, gdt, gpu, idt, serial, pit, keyboard,
                    pci.rs, nvme.rs
  memory/          pmm, paging (gained map_mmio()), heap
  scheduler/       preemptive Tcb-based scheduler; identity lives here
  fs/              vfs.rs, tmpfs.rs, elf.rs, initrd.rs,
                    partition.rs (GPT), ext2.rs (read+write)
  ipc/             default-deny channel registry
```

```
security/         seccomp, capabilities (scaffolded beyond
                  CAP_WRITE_SERIAL/CAP_DRAW)
kshell/           mostly unused — dispatch lives in main.rs
hal/              mod.rs, console.rs, input.rs, display.rs,
                  registry.rs, block.rs
compositor/       separate prototype — NOT kernel-runnable yet
build-iso.sh, run-qemu.sh
patch_N_*.py      one-off patch scripts, kept for history
```

Rule unchanged: add a directory/file when the code that justifies it exists, not before.

PART 4 — The Real Bugs Found and Fixed

A permanent record, kept in the same spirit throughout this project's history — every real problem encountered, however it was found, kept for the future rather than discarded once fixed. Earlier entries (bugs 1–31, spanning the original kernel bring-up, the HAL Phase A scheduler race, and the NVMe storage work) are condensed here; full narrative detail for those remains in this document's own prior revisions if ever needed, but the short form below is what matters going forward.

4.1 Condensed history (bugs 1–31)

Covers, in order: syscall stack-switching and noreturn handling; a hand-assembled ModRM byte error; W^X stub-poke ordering; EFER.NXE enablement; a PMM with no `free_frame()` at all (pure leak, fixed); MutexGuard held across a noreturn jump (permanently holding the run-queue lock); missing inline-asm clobbers (silent register corruption); a panic handler that could deadlock reporting its own panic; the dead Multiboot2 boot path's removal; a `limine` crate API mismatch (`actual_revision()` vs. an assumed, wrong method name) caught by reading the real installed crate source; the HHDM/MMIO mapping risk correctly investigated and resolved *before* shipping (base revision 3 does not blanket-map PCI MMIO the way revision 0 did); an under-sized `map_mmio()` register window missing the NVMe doorbell registers; and **bug #31**, a real, reproduced runtime regression — stale `ADMIN_QUEUE`/`IO_QUEUE` persistent-singleton state after `nvmeinittest` independently reset the physical controller out from under them, diagnosed correctly from a clean before/during/after evidence pattern and fixed with an explicit `invalidate_persistent_state()` call.

4.2 This session's additions (bugs 32–35) — the ext2 write-support arc

Bug #32 — kshell's default identity is not root, and every prior ext2 test accidentally never noticed. The first `ext2writetest` run failed every write-touching step with `PermissionDenied`. Root cause: `vfs::write()` checks the caller's *actual* identity against the target file's real on-disk owner/mode before ever reaching the filesystem backend — and every ext2 test fixture is root-owned (created via `sudo cp` on the host), while kshell's own default identity is not uid 0. This never surfaced on any read-only test because mode 644's "other" bits already grant read access regardless of caller identity; only a real *write* exposed it. Fixed by having write-touching tests save the caller's real identity, elevate to `(0, 0)` for the duration, and restore it afterward unconditionally — a pattern now used consistently across `ext2writetest`, `ext2growtest`, and `ext2deletetest`.

Bug #33 — a passing negative test can go stale when a later patch legitimately expands the boundary it's checking. Occurred *twice* in this session, both instances the same class of issue: `ext2writetest`'s "oversized write" check (originally 8192 bytes) started *passing* when patch_49 added real block-growth support, because 8192 bytes was no longer actually too large once growth into 12 direct blocks (12288 bytes) became legitimate. Separately, both `ext2mounttest`'s and `ext2writetest`'s "write to a nonexistent path" checks started *succeeding* when patch_50 added real file creation, because writing a new file is now supposed to work. Neither was a bug in the write/creation path itself — both write paths behaved correctly; the tests' own boundary assumptions were what needed updating. Fixed by retargeting each to the boundary that remains genuinely enforced (a write exceeding all 12 direct blocks; a write whose *parent directory* doesn't exist, since directory creation stays out of scope). Logged as a standing pattern: **when a patch legitimately expands what a system can do, immediately audit every existing negative test that checked the old boundary — a green run doesn't mean the test is still meaningful.**

Bug #34 — `insert_dir_entry()` initially couldn't reuse space freed by deletion. When file deletion (patch_51) was first designed, the existing `insert_dir_entry()` (written for patch_50's creation-only world) only ever looked at trailing slack past a *live* directory entry — it had no concept of a fully-freed (`ino == 0`) slot at all, meaning deleted space could never actually come back for reuse; every deletion would have permanently shrunk a directory's effective free capacity. Caught during design, before writing `delete_file()`, by reasoning through what `remove_dir_entry()` would actually leave behind (a first-in-block deletion has no preceding entry to merge into, so it must leave an `ino=0` marker) and checking whether the *existing* insertion code could recognize that shape. It couldn't. Fixed by adding an explicit branch: an `ino == 0` slot with `rec_len` big enough is claimed directly (no shrink/split needed, since there's no live entry to preserve), separately from the existing live-entry-slack-splitting logic.

Bug #35 — a directory-target permission check could mask the intended error. `ext2deletetest`'s negative case for "delete a directory" initially risked hitting `PermissionDenied` from `vfs::remove()`'s own up-front permission

check (against a real, host-created, root-owned `/subdir`) before `Ext2Filesystem::remove()` ever got a chance to return the intended `IsADirectory`. Caught in design review (the same reasoning as bug #32, applied proactively this time rather than discovered by a failing run) and fixed by elevating the whole test to root, the same established pattern.

4.3 Later additions this session (bugs 36-39) - indirect allocation, directory growth/deletion, crash-safety

Bug #36 - a real panic risk caught in design review before it ever ran. While extending write_file_in_place()'s growth path to reach the indirect block (patch_52), the pre-existing "defensive hole check" loop would have indexed past the end of the pointer array - and panicked - the first time growth reached into a not-yet-allocated indirect region, since the array only has 12 entries until the indirect block actually exists. Fixed by bounding the loop to the array's real current length before ever running the patch.

Bug #37 - a genuine kernel OOM from oversized test-code, not the filesystem. The first version of ext2writetest's "too big" threshold used 10,000,000 bytes - safely past the real capacity ceiling in the abstract, but the Vec::push loop building that buffer inside kshell itself exhausted the kernel's heap before the write path was ever reached, producing a real panic (OOM allocating 10000000 bytes). Fixed by recognizing that a kshell test's own buffer allocation shares the same limited heap the kernel needs, and re-deriving the threshold from this specific test image's actual known values (block_size=1024, giving a real cap of exactly 274,432 bytes) rather than a defensively-huge round number. Logged as a standing lesson: test-data sizing must respect the kernel's own heap budget, not just the filesystem boundary under test.

Bug #38 - the same "stale negative test" pattern, twice more. Once patch_52 (indirect growth) shipped, ext2writetest's oversized-write threshold needed re-deriving a second time (300,000 bytes, still safely below the new indirect-extended cap, had to become large enough to exceed it for real). Once patch_54 (directory deletion) shipped, ext2deletetest's directory-target negative check - which had correctly expected the generic IsADirectory before real directory-deletion dispatch existed - needed updating to expect the more accurate DirectoryNotEmpty, since a non-empty directory target now reaches delete_directory() for real instead of falling through to delete_file()'s generic rejection. Neither was a bug in the new capability; both were the test's own boundary assumptions going stale, the same standing pattern first logged with bug #33.

Bug #39 - two genuinely dangerous orderings, found by a deliberate crash-safety audit (patch_55), not by a failing test. delete_file() and delete_directory() both freed a target's data blocks BEFORE removing its directory entry - a crash in that gap would leave a live directory entry pointing at blocks the bitmap had already marked free, and if anything reallocated them before the stale entry was cleaned up, reading through it could return a different file's data. Separately, insert_dir_entry()'s directory-growth path linked a freshly allocated block into a directory's inode BEFORE initializing its content - since freed blocks are never zeroed, a crash in that gap could leave a directory block reachable while still holding stale garbage from whatever used it last, which read_dir_entries() would misparse as real directory-entry structure. Both fixed by reordering: remove-reference- before-free for deletion, initialize-before-link for directory growth - in each case turning "a crash here could corrupt something reachable" into "a crash here can only ever leak something unreachable." See Part 29's session log for the full audit reasoning, including every sequence checked and found already safe.

PART 5 — Established Workflow Conventions

Followed without exception across every session, including this one:

- Inline Python heredoc patch scripts with exact old/new string matching and a printed match-count check that aborts and writes nothing if the count isn't exactly 1 — never a blind `sed`/`patch`.
- Always verify current file content before writing a patch; grep for both new-code presence and old-code absence after every patch.
- Rebuild with the direct `cargo` command before trusting `build-iso.sh`'s own (historically error-swallowing) output.
- Test in QEMU always, first.
- The full hardware-verification structure ending in a Dell photo and a scaffolding revert, for anything claiming Dell-verified status.
- **New this session, generalized from the write-support work:** any patch script touching structural on-disk data (directory entries, bitmaps, inode records) gets its exact byte-offset math double-checked against the real on-disk format documentation before being written, the same discipline already applied to NVMe command layouts and GPT header fields — never inferred from a prior session's summary of the format.
- **Real disk-mutation discipline** (new this session, given how much of the ext2 write-support arc genuinely mutates the test image): every kshell test that performs a real, persistent write either (a) targets its own dedicated scratch file, never a shared fixture another test depends on, or (b) is fully self-contained and cleans up after itself (only `ext2deletetest` currently qualifies for (b) — everything else permanently mutates the test image in a documented, harmless way).

PART 6 — What's Next, In Order

The ext2/GPT/HAL storage arc (Phases B.1 and C, patches 33–51) is complete by this project's own QEMU standard. Immediate priorities, in order:

1. Dell hardware verification of the entire storage + filesystem write-support arc. Everything from `pcitest` through `ext2deletetest` is currently QEMU-only. Per the standing safety rule (Part 2.6), this requires a disposable external test drive before it can happen — not the Dell's internal drive again.

2. Networking — the next real roadmap milestone (per-process socket capability, a mandatory chokepoint with no silent fallback, Tails-derived design). Nothing has been started; no external reference material has been fetched yet, per this project's own standing rule that research happens per-subsystem, when that subsystem is actually reached. A candidate NIC driver scaffold (`rtl8139.rs`) exists in-tree from an earlier era but is currently unused and pre-dates a decision on `smoltcp` vs. a from-scratch stack — that decision should be made fresh, not assumed from the old scaffold.

3. Graphics / Compositor. Deferred until now specifically because it needed a real IPC layer for a client/compositor protocol — IPC has existed since early in this project, so this is unblocked whenever it's picked up, just not yet started for real (the existing "3 windows + taskbar" is a kernel-side placeholder, not the real compositor).

4. Package Manager — signature verification and sandboxing from day one, not started.

5. Linux Compatibility Layer — a small, audited syscall allowlist, not started.

6. Desktop — where the multi-repo/HAL/filesystem restructuring mentioned in Part 3.1 finally becomes worth doing, not started.

7. Hardening pass — ASLR, secure boot, deeper audit, not started.

If ext2 is ever revisited before Networking, the remaining items (Part 2.5) are sized, smallest to largest: directory growth (moderate — needs a real block-allocation call inside `insert_dir_entry()`'s failure path) → indirect-block allocation (moderate-to-large — needs the same growth logic extended past the 12-direct-block ceiling) → directory deletion (moderate — needs `./..` and link-count handling) → true crash-safety (large, cross-cutting, its own dedicated effort, should not be bundled with anything else).

PART 7 — How to Keep This Document Honest

Part 2 (ground truth) only changes when something is verified — not when it's written, not when it compiles, not when it works in QEMU alone if the claim is meant to be Dell-verified. Be skeptical of round-number completion percentages from any informal summary. Trust this document's item-by-item DONE/QEMU-only/not-started status instead, and trust a status claim only once real build/test output has been pasted back in support of it — every single DONE claim for the ext2 write-support arc in this document is backed by an actual pasted `kshell` transcript from this session, not an assumption. Keep this as the single authoritative document; don't let a second parallel doc start later.

PART 8 — Desktop/UI Vision (reference only, not current work)

Dark graphite/matte aesthetic with subtle cyan/blue neon edge lighting, performance-first compositing (cache blur, animate only changing regions, GPU over CPU, auto-degrade on battery/load). Differentiator: sits on top of the real security architecture already being built (per-process isolation, capabilities, VFS permissions, IPC) rather than being decorative UI bolted onto a conventional kernel. Not acted on now — last in the roadmap, deliberately.

PART 9 — Condensed Historical Session Logs (Parts 9–19)

Full narrative detail for these sessions exists in this document's own prior revisions. Condensed here to keep this consolidation focused; nothing below has changed status since it was originally logged.

CAP_DRAW + tmpfs ownership session. Closed two named weaknesses: `sys_draw` wasn't actually capability-gated, and tmpfs stamped every new file `uid=0/gid=0` regardless of the real caller. Both fixed, proven via dedicated tests (`drawtest`, `ownertest`), QEMU+Dell.

Module-bloat cleanup session. Removed 5,132 lines of dead code across 19 targets (10 modules, 8 unused drivers, one orphaned syscall dispatcher), verified via a full regression sweep, zero regressions.

Patch-workflow reference session. Fixed a stale `help` command gap (missing `drawtest`/`ownertest` in the printed list — the first instance of a recurring pattern that resurfaced several more times across this project's history). Established the step-by-step patch-writing and Dell-flashing reference parts.

HAL Phase A execution + scheduler race session. Implemented `Console`/`InputDevice`/`DisplayDevice` wrappers. Found and fixed a real, intermittent scheduler race: `switch_to()`'s deliberate immediate re-enqueue of the outgoing task meant a caller's naive "`yield_once()`-then-`reap()`" pattern could reap a task that had only been preempted, not actually finished. Fixed with a bounded wait-for-Zombie polling loop, applied to all five affected call sites.

HAL Phase B.1 (NVMe) session. PCI enumeration, real MMIO mapping (with the HHDM/base-revision investigation described in bug list §4.1), admin queue bring-up, Identify Controller/Namespace, I/O queues, real Read/Write, and the `BlockDevice` HAL abstraction. One real regression (bug #31, stale persistent-singleton state) found and fixed within the same session.

Environment migration + Dell NVMe incident (Part 19). Migrated from Windows/WSL2/Rufus to Omarchy Linux directly on the Dell. While validating real hardware write paths, `nvmeiotest`/`blockdevtest` wrote to LBA 0 of the Dell's *only* internal NVMe drive — its own protective MBR — because the "separate test hardware" turned out to be the same physical machine. Root-caused correctly (exactly one sector overwritten, drive otherwise intact, a standard `gdisk` protective-MBR rebuild would have been a zero-data-loss fix) but the person chose a drive replacement instead. This produced the standing safety rule in Part 2.6.

HAL Phase C (GPT + ext2, read-only) session. GPT partition parsing, a from-scratch read-only ext2 driver (deliberately not adapted from any existing GitHub implementation — every real candidate assumed a hosted environment this bare-metal kernel doesn't have), real VFS integration, and `mount_root()` wired into the actual boot sequence. Verified against independent ground truth created on the host via `sgdisk`/`mke2fs`, never round-tripped against the kernel's own writes. A genuine gap was found and fixed along the way: `fs::vfs::readdir()` didn't exist as a top-level free function at all before this session.

PART 20 — SESSION LOG: Multi-Level Path Resolution (patch_45, QEMU confirmed)

Closed the "root-directory lookups only" gap from the original read-only ext2 driver. `fs/ext2.rs` gained `resolve_path()`, walking a `/a/b/c` path directory-by-directory from `ROOT_INODE` via the existing `read_dir_entries()`, returning `NotADirectory` if a non-final segment isn't a directory. `lookup_in_root()` kept as a thin single-segment wrapper. The VFS adapter (`Ext2Filesystem::read/readdir/stat`) no longer rejects embedded `/` — real multi-level paths resolve through the ordinary `fs::vfs::` chokepoints, the same permission-checked path tmpfs already uses.

New kshell command `ext2pathtest`: two-level read + `readdir`, both negative cases (`NotADirectory` traversing through a file, `NotFound` for a genuinely missing nested path). Verified against ground truth created independently on the host. Also fixed the `help` command's stale list, which had `ext2test` duplicated five times instead of listing `ext2pathtest` — the same recurring gap class logged twice before in this project's history.

PART 21 — SESSION LOG: Singly-Indirect Block Reading (patch_46, QEMU confirmed)

Raised the ext2 read path's size cap from 12 direct blocks (`12*block_size`, ~12KB at this test image's 1024-byte block size) to `(12 + block_size/4) * block_size` by adding `block_pointers()`, a shared source of truth returning direct blocks plus — when `i_block[12]` is non-zero — every pointer from that one singly-indirect block. Both `read_dir_entries()` and `read_file()` now walk this combined list instead of each independently hardcoding `i_block[0..12]`.

New kshell command `ext2indirecttest`: a 20000-byte file (safely past the old 12KB cap), verified byte-for-byte against a computed pattern (`(i*7+3) & 0xFF`) rather than an embedded literal string, following the same convention `nvmeiotest` already established. `FileTooLargeForDirectBlocksOnly` renamed to `FileTooLarge` since the old name became inaccurate (doubly/triply indirect remain out of scope and still hit this error).

PART 22 — SESSION LOG: GPT CRC32 Validation (patch_47, QEMU confirmed)

Added a real CRC-32/ISO-HDLC implementation (`crc32()`, poly `0xEDB88320`) to `fs/partition.rs`. `read_gpt()` now validates both the header checksum (offset 16, computed over the first `HeaderSize` bytes with the checksum field itself zeroed, per the UEFI spec) and the partition-entry-array checksum (offset 88, computed over the actual bytes read). Two new error variants, `BadHeaderChecksum`/`BadPartitionArrayChecksum`, are hard errors on mismatch — a corrupted GPT is rejected now, not silently trusted. Verified against the project's own real `sgdisk`-created GPT; both checksums matched on the first attempt.

PART 23 — SESSION LOG: Write-In-Place (patch_48, QEMU confirmed)

The first real ext2 write capability. `fs/ext2.rs` gained `write_bytes_at()` (a sector-safe read-modify-write counterpart to `read_bytes_at()`, via `hal::registry::block_write()`), `allocated_prefix_capacity()` (counts an inode's consecutive already-nonzero block pointers from `i_block[0]`), and `write_file_in_place()` (overwrites an **existing** file within that capacity — `i_size` may grow or shrink freely within it, but no new block allocation happens in this patch). `Ext2Filesystem::write()` now calls into this instead of unconditionally returning `PermissionDenied`.

New kshell command `ext2writetest`, using a dedicated scratch file (`/writetest.bin`) rather than any shared fixture, since real `BlockDevice` writes are not ephemeral (Part 19.2). Verified: baseline check, a real content+shrink write round-tripped through an actual disk write and re-read, oversized-write rejection, write-to-nonexistent-path rejection, and a restore-to-original step confirmed idempotent.

Bug found during this patch's own verification (see Part 4 §4.2, bug #32): the first `ext2writetest` run failed every write step with `PermissionDenied` — kshell's default identity isn't root, and the target fixture is root-owned. Fixed with the save-elevate-restore identity pattern now used throughout the write-support arc. Also retargeted `ext2mounttest`'s existing write-denial assertion away from `hello.txt` (where a successful write would have permanently corrupted a shared fixture) to a path with no risk of accidental success.

PART 24 — SESSION LOG: Block Allocation for Growth (patch_49, QEMU confirmed)

Extended write support with real block allocation. `Ext2Superblock` gained `first_data_block`; `BlockGroupDescriptor` gained `block_bitmap` and `free_blocks_count`. New `allocate_one_block()` — a genuine block-bitmap scan across every block group on the filesystem (this test image has 8, not simplified to one), finding the first free bit, setting it, and decrementing both the group's and the superblock's free-block counts. `write_file_in_place()` gained a growth path: when a write exceeds current capacity, it allocates exactly as many new blocks as needed and writes their numbers into the inode's next empty **direct** slots (`i_block[0..12]` only — growth requiring indirect-block *allocation* returns `GrowthExceedsDirectBlocks`, explicitly deferred).

New kshell command `ext2growtest`. Verified across **7 consecutive fresh reboots**: the first exercised real allocation (10 → 1500 bytes, forcing a genuine second-block allocation, memcmp'd across the block boundary), every subsequent run correctly took the cheaper already-allocated-capacity path instead — exactly the predicted behavior.

Bug found afterward (Part 4 §4.2, bug #33, first instance): a regression sweep later revealed `ext2writetest`'s "oversized write" negative check (8192 bytes) now legitimately *succeeded*, since growth had expanded what's writable past that size. Not a bug in the write path — the test's own threshold was stale. Fixed by raising it to 200000 bytes, comfortably past the real 12-direct-block ceiling regardless of block size.

PART 25 — SESSION LOG: Real File Creation (patch_50, QEMU confirmed)

`allocate_one_inode()` — the inode-bitmap counterpart to block allocation. `insert_dir_entry()` — inserts a new directory entry by reusing **trailing slack** in an existing entry's `rec_len` (real ext2 directories routinely have this, since `mke2fs` pads the last entry per block to the block boundary); returns `DirectoryFull` if no block has room (directory growth stays out of scope). `create_file()` — allocates a fresh inode, writes a minimal record (mode `0o100644`, given uid/gid, size 0, one link, no data blocks yet), inserts its directory entry. Wired in via `Ext2Filesystem::write_owned()` — the method `vfs::write()` *always* calls — which now dispatches to either an overwrite of an existing file or a real create-then-write.

New kshell command `ext2createtest`: genuine creation on the very first ever run, correct fallback to ordinary overwrite on every run after (since deletion didn't exist yet at this point in the arc). Verified via both `read()` and `readdir()`.

Bug found afterward (Part 4 §4.2, bug #33, second instance): both `ext2mounttest`'s and `ext2writetest`'s "write to a nonexistent path" negative checks now legitimately *succeeded*, since creation made that a real, correct capability. Retargeted both to the boundary that remains genuinely enforced — a nonexistent *parent* directory (directory creation stays out of scope).

PART 26 — SESSION LOG: Real File Deletion (patch_51, QEMU confirmed) — closes the write-support arc

`fs/vfs.rs` gained `Filesystem::remove()` (default `PermissionDenied`, zero edits needed elsewhere) and a new `fs::vfs::remove()` free function. `fs/ext2.rs` gained `free_one_block()`/`free_one_inode()` (exact inverses of the allocation functions), `remove_dir_entry()` (the standard ext2 unlink technique — merges the deleted entry's

`rec_len` into the preceding entry in the same block, or leaves a fully-free `ino=0` slot if it was first in the block), and `delete_file()` (frees every data block including the indirect block itself, zeroes and frees the inode, removes the directory entry; rejects directories with `IsADirectory`).

Bug found in design, before it could ship (Part 4 §4.2, bug #34): the creation-era `insert_dir_entry()` had no concept of a fully-freed (`ino == 0`) slot at all — deleted space could never have come back for reuse. Fixed by adding an explicit branch recognizing and directly claiming such slots, in the same patch that introduced deletion.

New kshell command `ext2deletetest` — the only test in the entire arc that's fully self-contained and leaves zero permanent artifact: create → verify (read + `readdir`) → delete → verify genuinely gone (both ways) → **re-create under the same name** (the real proof of correctness — confirms the freed inode and dirent slot are genuinely reusable, not just marked-and-skipped) → two negative cases → final cleanup. A design-time permission-check risk (Part 4 §4.2, bug #35) was caught and fixed before the first run, by elevating the whole test to root. Verified twice consecutively with byte-identical output — genuinely idempotent.

Full regression sweep, ten commands, one boot session: `ext2deletetest` ×2, `ext2createtest`, `ext2growtest`, `ext2writetest`, `ext2mounttest`, `ext2test`, `ext2pathtest`, `ext2indirecttest`, `gpttest`, `roottest` — zero regressions. Every fixture from every prior patch (`hello.txt`, `subdir/nested.txt`, `bigfile.bin`, `writetest.bin`, `growtest.bin`, `createtest.bin`) confirmed exactly where it should be.

PART 15 — Recurring Mistakes Log (cumulative, keep appending)

Long-standing entries (wrong working directory, pasting multiple commands as one block, guessing a build invocation instead of reading the project's own config, assuming a Windows path exists without checking, trusting a terminal's echoed paste over a script's own printed result, a patch targeting a `match` statement that was actually an `else if` chain, guessing a crate API from memory instead of the real installed source, re-running a patch script against an already-patched file due to a substring-only match-count check) all remain valid and are not repeated here in full.

- **A passing negative test can go stale when a later patch legitimately expands the boundary it's checking.** Happened five separate times across this arc: `ext2writetest`'s oversized-write threshold (twice — once when growth shipped, once when indirect-block growth shipped), `ext2writetest` and `ext2mounttest`'s write-to-nonexistent-path checks (when creation shipped), and `ext2deletetest`'s directory-target check (when real directory-deletion dispatch shipped, expecting the old generic `IsADirectory` instead of the new, more accurate `DirectoryNotEmpty`). None were bugs in the new capability — the capability worked correctly; the test's assumption about what should still fail was what needed updating. Standing fix: whenever a patch legitimately expands what the system can do, immediately audit every existing negative test that checked the old boundary, rather than trusting a green regression sweep at face value.
- **Real writes to a root-owned test fixture, from a non-root default identity, silently pass on read but fail on write.** Cost a wasted round trip the first time (`ext2writetest`'s first run). Read-only tests never surface this, because a standard `0o644` mode already grants "other" read access regardless of caller identity — only a real write exposes the gap. Standing fix: any kshell test that performs a real write against a host-created (root-owned) fixture must save, elevate to `(0,0)`, and restore the caller's identity, unconditionally, covering every exit path.
- **`losetup -Pf` loop-device automount paths and `mount`/`umount` versus `udisksctl unmount` mismatches cost many round trips across this arc** when staging new ext2 test fixtures onto the real disk image via the host. Three distinct sub-problems, all traced back to the same root cause: `udisksctl` auto-mounting on its own schedule, uncoordinated with whatever the terminal was doing.
  1. `losetup -Pf` on this host tends to get auto-mounted by `udisksctl` at a path like `/run/media/<user>/<label>/`, not wherever a manual `mount` command targets — writing into that mounted path as a regular user fails with `Permission denied` (the ext2 root directory is `root:root` mode 644, since `mke2fs` ran as root) unless `sudo cp` is used for the actual file write, even though `sudo losetup`/`sudo mount` alone are not sufficient.
  2. The automount can race a manual `mount` command run immediately after `losetup -Pf`, producing the SAME device mounted at TWO paths simultaneously — writes and reads through the two mounts can see inconsistent, uncoordinated cached state (this genuinely corrupted a fixture down to 0 bytes once). Fix: after any double- mount, unmount the manual one and keep only udisks' own.
  3. The automount label can get a numeric suffix (`noxisatest1` instead of `noxisatest`) if udisks believes the label is already in use by a stale mount record — assuming path continuity from a prior session silently sends a copy nowhere. Standing fix, covering all three: always check `lsblk /dev/loopN` (or `mount | grep`) for the REAL current mountpoint immediately before copying anything, every single time, never assume continuity from a prior round — and if two mounts appear, drop the manual one and use only udisks' automount path.
- **Test-data sizing inside kshell must respect the kernel's own heap budget, not just the filesystem boundary under test.** A "safely past the real cap" test buffer sized at 10,000,000 bytes caused a genuine kernel panic (`OOM allocating 10000000 bytes`) building the buffer in kshell itself, before the write path it was meant to test was ever reached. Standing fix: derive oversized-test thresholds from the specific test image's actual known values (e.g. its real block_size), not a defensively huge round number — the buffer has to actually fit in the kernel's heap to prove anything.

PART 27 — SESSION LOG: Indirect-Block Allocation, Directory Growth/Deletion, and a Crash-Safety Audit (patches 52-55, QEMU confirmed)

Closes every remaining named item from the original ext2 write-support arc, across four patches in one continued session.

patch_52 — indirect-block allocation. Extended `write_file_in_place()`'s growth path past the 12-direct-block ceiling: when growth needs more blocks than remaining direct slots can hold, it now allocates the indirect block itself (if not already present), zeroes it, links it into the inode, and writes further pointers directly into it. The old `GrowthExceedsDirectBlocks` error retired in favor of the read side's existing `FileTooLarge`, now the real ceiling for both directions. New test `ext2indirectgrowtest` (10 → 15000 bytes, forcing real allocation of all 12 direct blocks plus the indirect block and its own entries) passed on the first attempt — but only after a bounds-check bug that would have panicked on the very first indirect allocation was caught and fixed in design review before ever running it (bug #36).

patch_53 — directory growth. Extracted `grow_inode_to_block_count()` out of `write_file_in_place()`'s own logic into a primitive shared with directory growth — `insert_dir_entry()` now falls back to growing the directory by one block (initialized as one giant free entry) when no existing block has room, retrying afterward. New test `ext2dirgrowtest`: 100 small files forced into `/subdir`, a real second directory block allocated and verified, all cleaned up afterward.

patch_54 — directory deletion. `delete_directory()` removes an empty directory (refusing non-empty ones with the new `DirectoryNotEmpty` error), freeing its blocks and inode, removing its entry from its parent, decrementing the parent's link count and the block group's `bg_used_dirs_count` — a field never touched before. `Ext2Filesystem::remove()` now dispatches by target type. New test `ext2rmdirtest`, requiring a host-staged empty directory since this driver still has no `mkdir` — the only way to prove genuine successful deletion, not just the refusal path.

patch_55 — the crash-safety audit, done properly rather than skipped. A full pass over every multi-step write sequence built across patches 45-54, checking what a power loss mid-sequence could leave behind. Found two categories: sequences already safe by construction (documented explicitly rather than left implicit — see each function's own doc comment), and two genuinely dangerous orderings, both reversed:

- `delete_file()` / `delete_directory()` now remove the directory entry *first*, free resources *after* — the old order could leave a live entry pointing at blocks the bitmap already called free, risking a reader seeing a different file's data through a stale entry if those blocks were reallocated before cleanup. New order: a crash after entry removal can only ever leak, never dangle.
- `insert_dir_entry()`'s directory-growth path now allocates a raw block, writes its full content, and only then links it into the inode — the old order could leave a directory block reachable (and parsed as structure by `read_dir_entries()`) before it held valid content, since freed blocks are never zeroed. New order: a crash before linking just leaks an unlinked block.

Explicitly not attempted: true atomicity or a journal (still several separate writes, not one transaction — a crash can still interrupt a single bitmap-bit write or leave a count stale, both cosmetic and fsck-fixable, never dangerous) or real crash-injection testing (would need an actual QEMU restart-mid-write harness, its own separate effort). Verified by code-review audit plus the full regression suite, not by simulating real power loss.

Verification across this whole session: thirteen-command full regression sweep, one boot session, zero regressions — every fixture from every earlier patch (`hello.txt`, `subdir/nested.txt`, `bigfile.bin`, `writetest.bin`, `growtest.bin`, `createtest.bin`, `indirectgrowtest.bin`) confirmed intact throughout, on top of the widest-blast-radius change (patch_55) in the entire arc.

PART 28 — Patch Index (cumulative)

A quick lookup from patch number to what it did, across every session. Patches 1–22 are original kernel/isolation/capability work (see condensed Part 9); patches 23–32 are HAL Phase A and its scheduler-race fix; patches 33–38 (+37b) are HAL Phase B.1 (NVMe); patches 41–44 are HAL Phase C (GPT + read-only ext2 + VFS integration + `mount_root()`); **patches 45–55 are the complete ext2 write-support arc, now fully closed:**

Patch	What it added
45	Multi-level ext2 path resolution (`/a/b/c`)
46	Singly-indirect block reading (files past 12 direct blocks)
47	GPT header + partition-array CRC32 validation
48	ext2 write-in-place for an existing file, within already-allocated blocks
49	Real block allocation — growth into new direct blocks via block-bitmap scan
50	Real file creation — inode allocation + directory-entry insertion
51	Real file deletion — block/inode freeing + directory-entry removal with slot reuse
52	Indirect-block allocation — growth past 12 direct blocks, for real
53	Directory growth — allocating a directory an additional block when out of slack
54	Directory deletion — removing an empty directory, with correct link-count/used-dirs bookkeeping
55	Crash-safety ordering audit — reordered the two genuinely dangerous sequences

PART 29 — Executive Summary: What's Done, What Broke and Got Fixed, What's Left

(This section exists specifically to answer, at a glance: what's completed across the whole project, what bugs were found and fixed, and what's genuinely still remaining.)

What's fully done (proven QEMU+Dell, stable across many sessions)

Core kernel bring-up (boot, memory, CPU, syscalls, ring 3), the complete isolation/security milestone (per-process page tables, W^X, guard pages, capabilities, VFS permissions, default-deny IPC, memory scrubbing, audit logging, real teardown), generic process spawning with per-process identity/resources, multiple concurrent processes, real preemptive scheduling, and the HAL Phase A device wrappers (console/input/display).

What's done, QEMU-verified, Dell verification outstanding

The entire NVMe storage stack (PCI enumeration through real Read/Write I/O and the `BlockDevice` HAL abstraction), GPT partition parsing with real CRC32 validation, and the **complete ext2 filesystem driver**: read-only capabilities (multi-level paths, indirect-block reading, VFS integration, automatic boot-time root mount) plus the full write-support arc — real write-in-place, block allocation and growth including into the indirect block, file creation and deletion, directory growth and deletion, and a genuine crash-safety ordering audit — all independently verified with dedicated kshell tests and repeated full cross-test regression sweeps showing zero interference between any of them.

What bugs/errors were found and fixed (cumulative highlights)

Beyond the many kernel-level bugs fixed in earlier sessions (a genuine scheduler race between preemption and reaping, a real PMM frame leak, a stale-cache bug after an independent NVMe controller reset, and others — see Part 4 §4.1), the ext2 write-support arc's own real issues (Part 4 §4.2–4.3, bugs 32–39) were: a never-before-exposed identity/permission gap that only a real write could trigger; the "stale negative test" pattern recurring five separate times as later patches legitimately expanded system capability; two design-time catches (directory-entry insertion originally couldn't reuse space freed by deletion; a directory-deletion negative test could have hit the wrong error via a permission check) fixed before they ever shipped; a real panic-risk bounds-check bug caught in review before it ran; a genuine kernel OOM from a test buffer too large for the kernel's own heap; and, found by a deliberate audit rather than a failing test, two genuinely dangerous write orderings in file/directory deletion and directory growth — both reordered so a crash can only ever leak a resource, never leave a dangling reference or expose unparsed garbage as real structure.

What's genuinely still remaining

Immediate: Dell hardware verification of the entire storage + filesystem arc, blocked on acquiring a disposable external test drive per the standing safety rule.

Roadmap, in order, none started: Networking (per-process sockets, mandatory chokepoint) → Graphics/Compositor (unblocked by IPC, just not built) → Package Manager → Linux Compatibility Layer → Desktop → Hardening pass (ASLR, secure boot, deeper audit).

Ext2, if ever revisited (none of these block the roadmap): directory creation (`mkdir` — every directory-needing test still relies on a host-staged one), true atomicity/journaling beyond the ordering-safety patch_55 already provides, and real crash-injection testing via an actual QEMU restart-mid-write harness. All deliberately named and deferred, not forgotten.

Long-term, separate track: the "Wisdom AI" assistant concept, and the Desktop/UI vision (Part 8) — both deliberately untouched until the kernel/security/filesystem foundation is further along.

— End of consolidated document. This replaces every prior revision as the single authoritative reference, per this project's own Part 7 rule. Paste this whole document at the start of any new session.